

Resume-Job Description Matching (Similarity + Classification)

Problem:

Build a system that evaluates how well a candidate's resume matches a job description, useful for HR/recruitment automation.

Step-by-Step Plan

Step 1: Problem Framing

- Task 1: Calculate similarity between resumes and job descriptions.
 - Task 2: Classify into "match" or "no-match."
 - Goal: Automate shortlisting of candidates for recruiters.
-

Step 2: Dataset Preparation

- Use **Job Postings Dataset** (publicly available).
 - Simulate candidate resumes aligned with job postings.
 - Create labeled pairs: (Resume, Job Description, Match/No-Match).
-

Step 3: Text Preprocessing

- Clean text: lowercase, remove stopwords, punctuation, special characters.
 - Lemmatization/stemming for better normalization.
 - Convert text into numerical representation (TF-IDF / Doc2Vec).
-

Step 4: Feature Engineering

- Extract features from resumes and job descriptions.
 - Compute **Cosine Similarity** between embeddings → similarity score.
 - Combine similarity score with other features (like job title match, keyword overlap).
-

Step 5: Model Design

- **Classification Task:** Match vs No-Match.

- Try multiple classifiers:
 - **Baseline:** Logistic Regression / SVM.
 - **Deep Learning:** ANN (Dense Layers with embeddings).
-

Step 6: Training & Evaluation

- Split dataset into train/test sets.
 - Evaluate with **Accuracy, Precision, Recall, F1-score**.
 - Track performance for both similarity scoring and classification.
-

Step 7: MLflow Integration

- Log models, hyperparameters, and metrics.
 - Compare TF-IDF vs Doc2Vec embeddings.
 - Track experiments for similarity-only vs classification.
-

Step 8: Airflow Pipeline

- Automate pipeline:
 1. Ingest resumes + job descriptions.
 2. Preprocess text.
 3. Train models.
 4. Evaluate + push results to MLflow.
 5. Deploy updated model if performance improves.
-

Step 9: Deployment Layer (Docker + Kubernetes)

- Package model into a **Docker container**.
- Expose REST API (FastAPI/Flask):
 - Input: Resume text + Job description text.
 - Output: Match/No-Match + Similarity Score.
- Deploy on Kubernetes cluster for scaling.

Step 10: End-to-End Demonstration

- Show recruiter uploading resumes & job description.
- System returns:
 - Similarity score.
 - Classification: Match / No-Match.
- Highlight MLflow dashboard for model performance tracking.
- Demonstrate Airflow DAG automation.